

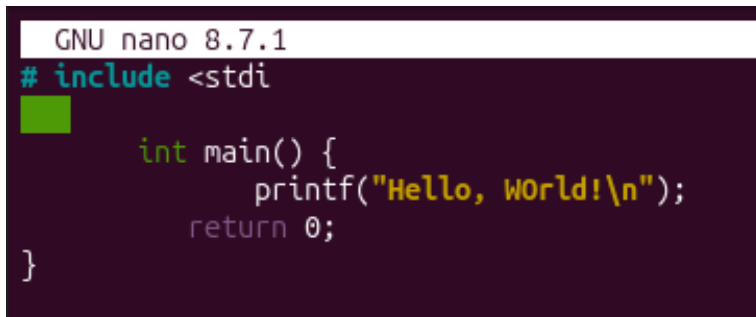
Table of Contents

Hello World Program	3
Summary	3
GCC Installation and Version Check	3
Summary	3
Four Stages of Compilation	4
Summary	4
File Type Check	4
Summary	4
Assembly Output	4
Summary	5
Return Values	5
Knowledge Check Questions	5
Q1. Purpose of the #include directive	5
Q2. Difference between the four compilation stages	5
Q3. Meaning of the return value from main()	5
Q4. Program that inputs and displays an integer	6
Q5. GCC commands for each stage	6
Q6. Why C is called a compiled language	6
Lab Tasks Completed	6

Table of Figures

Figure 1: hello.c opened in nano	3
Figure 2: Compile error, fix, and program output	3
Figure 3: build-essential already installed	3
Figure 4: GCC version check	4
Figure 5: Running all four compilation stages separately	4
Figure 6: file hello.o output	4
Figure 7: Contents of hello.s	5
Figure 8: Exit status check with echo \$?	5

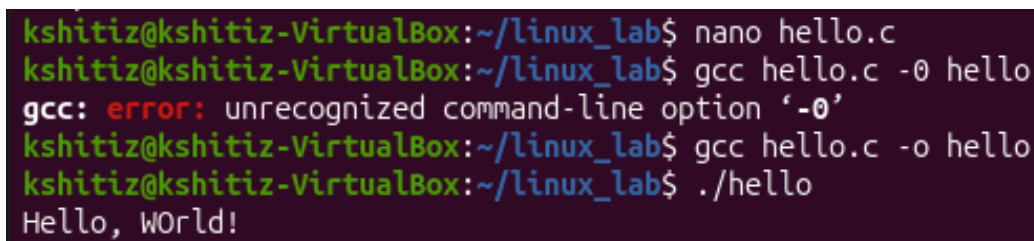
Hello World Program



```
GNU nano 8.7.1
# include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

[*Figure 1: hello.c opened in nano*](#)



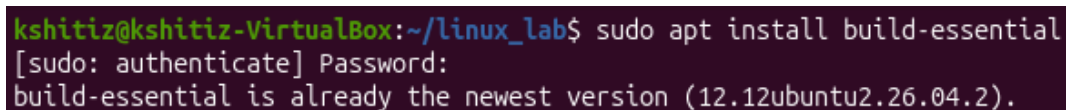
```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ nano hello.c
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc hello.c -0 hello
gcc: error: unrecognized command-line option '-0'
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc hello.c -o hello
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./hello
Hello, World!
```

[*Figure 2: Compile error, fix, and program output*](#)

Summary

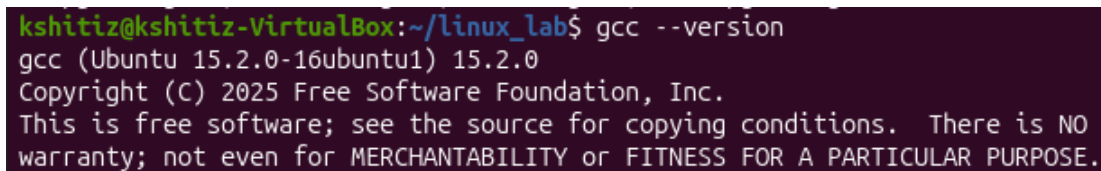
I wrote a simple C program called `hello.c` using the nano editor. It uses the header file `stdio.h`, a main function, and a `printf` statement that prints “Hello, World!” to the screen, then returns 0. When I first tried to compile it, it printed Hello, World! on the screen, which means the program worked.

GCC Installation and Version Check



```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
```

[*Figure 3: build-essential already installed*](#)



```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

[*Figure 4: GCC version check*](#)

Summary

I checked if the build-essential package was installed by running `sudo apt install build-essential`. It showed that build-essential was already the newest version, so nothing

new had to be installed. I also checked the GCC version using `gcc --version`. It showed GCC (Ubuntu 15.2.0-16ubuntu1), which confirmed that compiler was ready to use.

Four Stages of Compilation

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc -E hello.c -o hello.i
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ls
hello  hello.c  hello.i
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc -S hello.i -o hello.s
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ls
hello  hello.c  hello.i  hello.s
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc -c hello.s -o hello.o
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ls
hello  hello.c  hello.i  hello.o  hello.s
kshitiz@kshitiz-VirtualBox:~/linux_lab$ gcc hello.o -o hello
kshitiz@kshitiz-VirtualBox:~/linux_lab$ ls
hello  hello.c  hello.i  hello.o  hello.s
```

[*Figure 5: Running all four compilation stages separately*](#)

Summary

I had run the four GCC stages step by step. Preprocessing produced `hello.i`, compiling generated `hello.s`, assembling created `hello.o`, and finally linking combined everything into the executable `hello`. Each step was confirmed with `ls` showing the new file created.

File Type Check

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
kshitiz@kshitiz-VirtualBox:~/linux_lab$
```

[*Figure 6: file hello.o output*](#)

Summary

After creating `hello.o`, I checked what type of file it is using the `file hello.o` command. The result showed it is an ELF 64-bit object file.

Assembly Output

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ cat hello.s
        .file    "hello.c"
        .text
        .section      .rodata
.LC0:
        .string "Hello, world!"
        .text
        .globl  main
        .type   main, @function
```

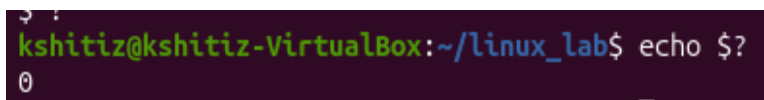
[Figure 7: Contents of hello.s](#)

Summary

I also looked inside hello.s using `cat hello.s`. It shows the text “Hello, WOrld!” stored under the `.rodata` section, and the main function is marked as global. This is the assembly code stage. This is readable but not machine level code.

Return Values and Exit Status

After running the program, I checked this using `echo $?`. It printed 0, which confirms the program returned 0 and finished without errors.

A terminal window with a dark purple background. The prompt is 'kshitiz@kshitiz-VirtualBox:~/linux_lab\$'. The command 'echo \$?' has been entered, and the output '0' is displayed on the next line.

```
kshitiz@kshitiz-VirtualBox:~/linux_lab$ echo $?  
0
```

[Figure 8: Exit status check with echo \\$?](#)

Knowledge Check Questions

Q1. Purpose of the #include directive

The `#include` directive copies a header file into the program so that we can use built-in functions like `printf()` and `scanf()`.

Q2. Difference between the four compilation stages

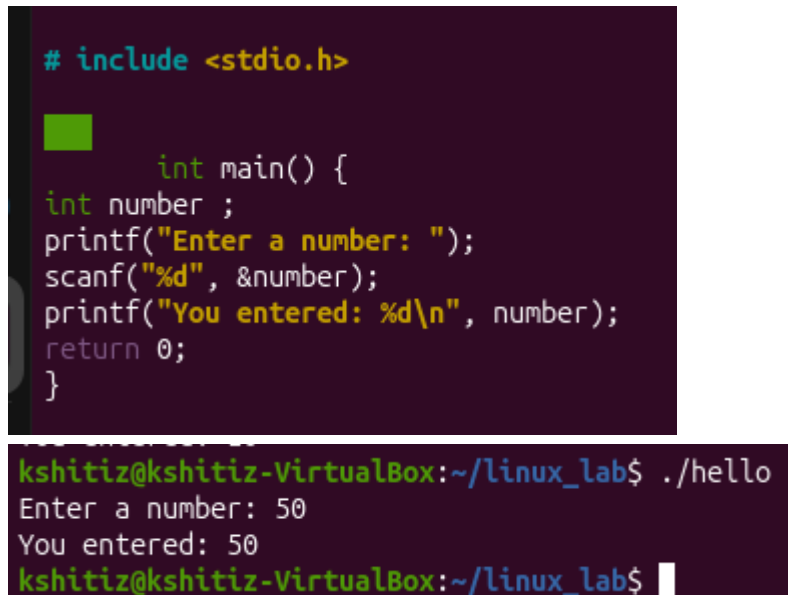
Preprocessing makes a `.i` file, Compiling makes a `.s` file, assembling makes a `.o` file and linking makes the final executable.

Q3. Meaning of the return value from main()

A return value of 0 means success, while any other value gives error signal.

Q4. Program that inputs and displays an integer

Below is a simple program that asks the user for a number and then prints it back:



```
# include <stdio.h>

int main() {
    int number ;
    printf("Enter a number: ");
    scanf("%d", &number);
    printf("You entered: %d\n", number);
    return 0;
}

kshitiz@kshitiz-VirtualBox:~/linux_lab$ ./hello
Enter a number: 50
You entered: 50
kshitiz@kshitiz-VirtualBox:~/linux_lab$
```

Figure 8: Program that inputs and displays integer

Q5. GCC commands for each stage

Gcc -E preprocesses, gcc -S compiles to assembly, and gcc -c creates an object file.

Q6. Why C is called a compiled language

Because GCC translates source code to machine code before running unlike interpreted language that runs line by line.

Lab Tasks Completed

In this lab I compiled hello.c with return 0 and checked the exit status using echo \$? . I also changed the program to return 1 and saw the exit status change. I wrote a program that takes user input and displays it back. I ran all four compilation stages separately and confirmed each output file was created. Finally, I compiled and ran a program that returns the integer value 50.